Calculadora II

Diseño e implementación de la interfaz con JavaFX

Calculadora de escritorio que se maneja con el ratón y con el teclado

Eva Gallardo Romero
Desarrollo de Interfaces
29 de octubre de 2026

Índice

1 Introducción **2**
1.1 Requisitos del enunciado . . . 2
1.2 Estructura del proyecto . . . 2
2 Sketch **3**
3 Wireframe **3**
4 Prototipo **4**
5 Estilo visual **5**
5.1 La idea . . . 5
5.2 Colores . . . 5
5.3 Tipografía y formas . . . 6
5.4 Por qué es así . . . 6
6 Implementación con JavaFX **6**
6.1 Modelo, vista y controlador . . . 6
6.2 La vista: calculadora.fxml . . . 6
6.3 El modelo: Calculadora . . . 6
6.4 El controlador: CalculadoraController . . . 6
6.5 Uso con el teclado . . . 8
6.6 La clase App . . . 9
6.7 Funcionamiento . . . 9
6.8 Limitaciones . . . 10
7 Diferencias respecto al prototipo **10**
8 Pruebas con JUnit **11**
8.1 Qué se prueba . . . 11
8.2 Cómo se ejecutan . . . 11
9 Documentación Javadoc **11**
10 Código desarrollado **12**
10.1 App.java . . . 12
10.2 Calculadora.java . . . 13
10.3 CalculadoraController.java . . . 13
10.4 calculadora.fxml . . . 16
10.5 CalculadoraTest.java . . . 18

1. Introducción

En esta práctica he diseñado e implementado la interfaz gráfica de una calculadora con JavaFX. La calculadora hace las cuatro operaciones básicas (suma, resta, multiplicación y división), permite encadenar varias operaciones seguidas y tiene un botón para borrar. Todo está en una sola ventana: arriba hay una pantalla donde se ven los números y el resultado, y debajo los botones.

Esta segunda versión parte de la calculadora de la tarea *006 – Calculadora* y añade el **uso con el teclado**: los números, las operaciones, el punto decimal y el resultado se pueden introducir con las teclas, y el botón de borrar tiene asignada la combinación **Ctrl + C** (sección 6.5).

1.1. Requisitos del enunciado

El enunciado de la primera tarea pide una calculadora con unos botones concretos y un campo de texto, y el de esta segunda tarea añade su uso con el teclado. El cuadro 1 resume cada requisito y dónde se cumple.

Requisito	**Cómo se cumple**
Botones de los números	Diez botones, del `0` al `9`, más el punto decimal.
Operaciones aritméticas	Cuatro botones: +, −, × y ÷.
Botón para obtener el resultado	Botón =.
Botón para borrar	Botón `C`.
Campo de texto con los números y el resultado	`Label` en la parte superior, que solo muestra texto (sección 6.2).
Uso con las teclas numéricas	Teclas `0`–`9` del teclado normal y del numérico (sección 6.5).
Uso con las teclas de las operaciones	Teclas +, −, `*` y `/`, además de Enter para el resultado (sección 6.5).
Combinación de teclas para borrar	**Ctrl + C** hace lo mismo que el botón `C` (sección 6.5).
Pruebas de las operaciones	Cinco pruebas con JUnit (sección 8).
Documentación Javadoc	Comentarios en todas las clases y métodos (sección 9).

Cuadro 1: Requisitos del enunciado y dónde se cumplen.

1.2. Estructura del proyecto

Separé el código siguiendo el patrón modelo–vista–controlador. El cuadro 2 muestra qué hace cada archivo.

Archivo	Parte	Contenido
`calculadora.fxml`	Vista	Pantalla y botones.
`CalculadoraController.java`	Controlador	Recibe los clics y las teclas y actualiza la pantalla.
`Calculadora.java`	Modelo	Las cuatro operaciones.
`App.java`	Arranque	Carga la vista, muestra la ventana y conecta el teclado con el controlador.
`CalculadoraTest.java`	Pruebas	Pruebas JUnit del modelo.

Cuadro 2: Archivos del proyecto.

2. Sketch

Empecé dibujando la calculadora a mano en papel. Me sirvió para decidir cómo repartir el espacio antes de pensar en colores o tipografías: la pantalla arriba, ocupando todo el ancho, y los botones debajo en una cuadrícula.

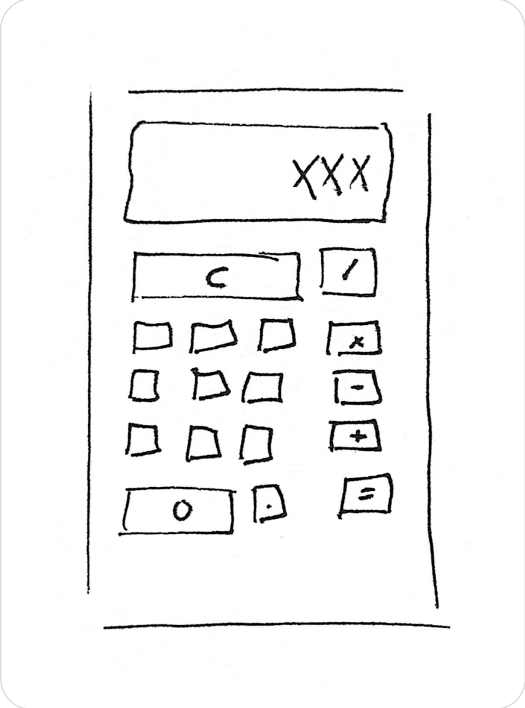


Figura 1: Sketch de la calculadora.

3. Wireframe

Después pasé el sketch a Figma en escala de grises, sin color ni tipografía exacta, para centrarme solo en la estructura.

Las decisiones principales fueron:

- **Pantalla arriba, botones abajo.** La pantalla ocupa todo el ancho y los números se alinean a la derecha, como en una calculadora normal.
- **Cuadrícula de cuatro columnas y cinco filas.** Los dígitos van a la izquierda y las operaciones forman la columna de la derecha, con el = en la esquina inferior derecha.
- **Dos botones más anchos.** El botón `C` ocupa tres columnas y el `0` ocupa dos, para que el borrado y el cero sean fáciles de pulsar.

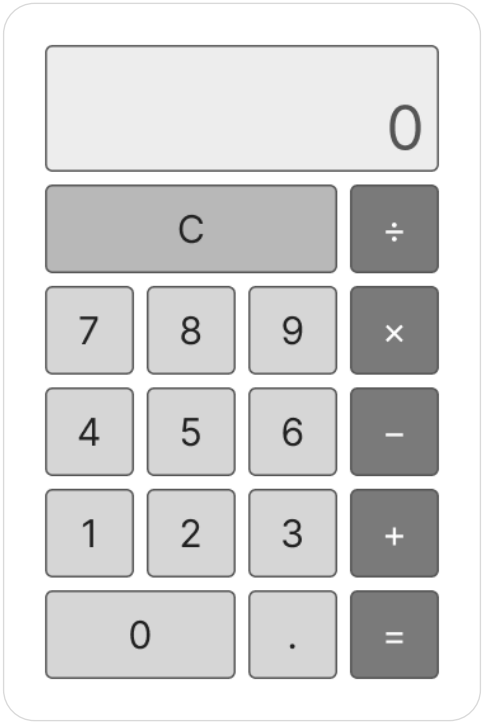


Figura 2: Wireframe de la calculadora.

4. Prototipo

Por último apliqué el estilo visual al wireframe. Como la calculadora tiene una sola ventana, aquí no hay navegación entre pantallas.

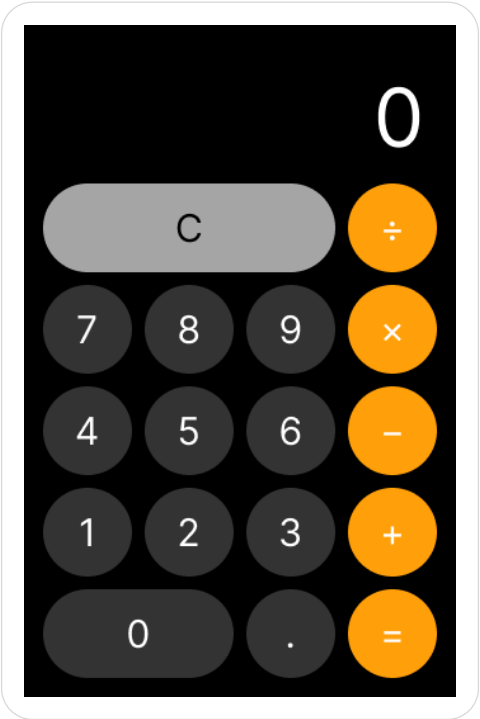


Figura 3: Prototipo de la calculadora.

5. Estilo visual

5.1. La idea

Una calculadora se usa muchas veces seguidas y sin pensar, así que quise un estilo claro y sin distracciones: fondo negro, botones redondos y un solo color vivo para las operaciones.

5.2. Colores

	Color	Código	Uso
	Negro	`#000000`	Fondo de la ventana y de la pantalla.
	Gris oscuro	`#333333`	Botones de los números y del punto.
	Gris claro	`#A5A5A5`	Botón `C` (con texto negro).
	Naranja	`#FF9F0A`	Operaciones y botón =.
	Blanco	`#FFFFFF`	Texto de la pantalla y de los botones.

Cuadro 3: Colores principales.

5.3. Tipografía y formas

Para el texto uso una tipografía sin serifa, clara y neutra. Los botones son del mismo alto y ancho salvo el de C y 0, más anchos, todos con los extremos redondeados.

5.4. Por qué es así

- **El color tiene un significado.** El naranja es solo para las operaciones y el resultado, el gris claro para borrar y el gris oscuro para los números. Así se encuentra cada tipo de botón de un vistazo.
- **Los botones se reconocen por la forma.** Los más anchos son los dos que más se usan: borrar y el cero.
- **Cada cosa está siempre en el mismo sitio.** Las operaciones están en la columna de la derecha y el resultado (=) en la esquina inferior.

6. Implementación con JavaFX

6.1. Modelo, vista y controlador

Separé la parte que calcula de la parte que se ve. El modelo (`Calculadora`) solo sabe sumar, restar, multiplicar y dividir, y no conoce ningún botón ni ventana. Esto permite probarlo con JUnit sin abrir la interfaz. La vista (`calculadora.fxml`) solo describe cómo se ve, y el controlador conecta las dos.

6.2. La vista: calculadora.fxml

La ventana es un `VBox`, dentro hay dos elementos:

- **La pantalla.** Un `Label`, alineado a la derecha y con el texto inicial `0`. Tiene `maxWidth=Ïnfinity"` para que ocupe todo el ancho de la ventana; si no, solo mediría lo que mide el número y no se pegaría a la derecha.
- **Los botones.** Un `GridPane` de cuatro columnas y cinco filas. `C` ocupa tres columnas (`columnSpan`) y `0` ocupa dos. Los números siguen el orden del prototipo, con `7 8 9` en la fila de arriba.

Cada botón tiene su evento `onAction`. Los botones de operación llevan además un atributo `userData` con el símbolo que entiende el programa (`+`, `-`, `*` o `/`), porque los símbolos que se ven en el botón (× y ÷) no sirven para calcular.

6.3. El modelo: Calculadora

6.4. El controlador: CalculadoraController

El controlador guarda el estado de la calculadora en cuatro atributos, además de la referencia a la pantalla y de la combinación de teclas para borrar:

Método	Operación	Observaciones
`sumar(a, b)`	$a + b$	
`restar(a, b)`	$a - b$	
`multiplicar(a, b)`	$a \times b$	
`dividir(a, b)`	$a \div b$	Lanza `ArithmeticException` si $b = 0$.

Cuadro 4: Métodos del modelo. Todos reciben y devuelven `double`.

Atributo	Tipo	Para qué sirve
`pantalla`	`Label`	Pantalla de la vista.
`calculadora`	`Calculadora`	El modelo, que hace las cuentas.
`primerNumero`	`double`	Número escrito antes de elegir la operación.
`operacion`	`String`	`+`, `-`, `*`, `/` o vacío si no hay ninguna.
`numeroNuevo`	`boolean`	Si es `true`, el siguiente número empieza de cero.
`combinacionBorrar`	`KeyCombination`	La combinación **Ctrl + C**, que borra la pantalla.

Cuadro 5: Atributos del controlador.

Tiene un método por cada tipo de botón, uno para el teclado y varios auxiliares. Para no repetir código, la lógica de escribir un dígito y de elegir una operación está en dos métodos que usan tanto los botones como el teclado:

Operaciones encadenadas

Al pulsar un operador, si ya había otro pendiente y se ha escrito un segundo número, `elegirOperacion` llama primero a `pulsarIgual` para calcular lo que se lleva. Así, $3 + 4 + 5$ da 12 sin necesidad de pulsar = entre medias:

Pulsación	Qué pasa	Pantalla
`3`	Se escribe el 3.	`3`
+	Guarda `primerNumero = 3` y `operacion = `"`.	`3`
`4`	Empieza un número nuevo.	`4`
+	Hay una suma pendiente: calcula $3 + 4$ y guarda `primerNumero = 7`.	`7`
`5`	Empieza un número nuevo.	`5`
=	Calcula $7 + 5$.	`12`

Cuadro 7: Ejemplo de operaciones encadenadas.

Redondeo de decimales

Un `double` no puede guardar exactamente algunos decimales, por eso 0,1+0,2 da `0.30000000000000004`. El método `formatear` multiplica el resultado por 10^8, lo redondea con `Math.round` y lo vuelve

Método	Botón	Qué hace
`pulsarNumero`	`0`–`9`	Saca el dígito del texto del botón y llama a `escribirDigito`.
`pulsarPunto`	`.`	Añade el punto decimal solo si el número aún no tiene uno.
`pulsarOperacion`	$+ - \times \div$	Saca el símbolo del `userData` del botón y llama a `elegirOperacion`.
`pulsarIgual`	$=$	Llama al modelo con un `switch` y muestra el resultado, o `Error` si se divide entre cero.
`pulsarBorrar`	`C`	Vuelve al estado inicial: pantalla a `0` y sin operación.
`teclaPulsada`	(teclado)	Recibe cada tecla pulsada y llama al método que corresponde (sección 6.5).
`escribirDigito`	—	Añade el dígito a la pantalla, hasta 10 caracteres; si es un número nuevo, lo sustituye.
`elegirOperacion`	—	Si había una operación pendiente, la calcula primero. Después guarda el número de la pantalla y la operación elegida.
`formatear`	—	Muestra los enteros sin decimales (`8` en vez de `8.0`) y redondea el resto a 8 decimales.

Cuadro 6: Métodos del controlador.

a dividir, de modo que se muestra `0.3`. Del mismo modo, $1 \div 3$ se muestra como `0.33333333`.

6.5. Uso con el teclado

El cuadro 8 muestra las teclas que entiende la calculadora. Cada una llama al mismo método que su botón, así que el resultado es idéntico se use el ratón o el teclado.

Tecla	Acción
`0`–`9` (teclado normal o numérico)	Escribe el número.
`+` `-` `*` `/`	Elige la operación.
`.` o `,`	Punto decimal.
Enter o $=$	Muestra el resultado.
Ctrl + C	Borra, igual que el botón `C`.

Cuadro 8: Teclas de la calculadora.

Dónde se escucha el teclado

En `App` se registra un manejador con `escena.setOnKeyPressed(...)`. Se pone en la **escena** y no en un botón porque en JavaFX los eventos suben desde el nodo donde se originan hasta la raíz (fase de *burbuja*). Así la escena recibe todas las teclas, esté donde esté el foco. Además,

los botones tienen `focusTraversable="false"`, para que ninguno se quede con el foco y, por ejemplo, Enter no pulse un botón por error.

El manejador solo pasa el evento al controlador, al método `teclaPulsada`. Para tener el controlador, `App` usa el `FXMLLoader` con el que carga la vista: `loader.getController()`.

Cómo se reconoce cada tecla

`teclaPulsada` comprueba las teclas en este orden:

1. **La combinación para borrar.** Se crea una sola vez, como atributo:

   ```
   new KeyCodeCombination(KeyCode.C, KeyCombination.CONTROL_DOWN)
   ```

   y se comprueba con `combinacionBorrar.match(evento)`, que solo es cierto si se pulsa la `C` con Ctrl a la vez. Va la primera para que ninguna otra condición se adelante.

2. **Los símbolos con Mayúsculas.** En el teclado español, `/` se escribe con Mayús + 7, `*` con Mayús y la tecla +, y = con Mayús + 0. Si `evento.isShiftDown()` es cierto, se tratan esas tres teclas y el resto se ignora, para que Mayús + 7 no escriba un 7.

3. **El resto de teclas.** Un `switch` sobre `evento.getCode()`, que devuelve un `KeyCode`. Cada número tiene dos códigos, `DIGIT7` en el teclado normal y `NUMPAD7` en el numérico, y los dos llevan al mismo `case`. Igual pasa con las operaciones: `ADD` (numérico) y `PLUS` (normal) eligen la suma.

6.6. La clase App

`App` es la clase de arranque. Carga `calculadora.fxml` con un `FXMLLoader`, recoge su controlador y registra en la escena el manejador del teclado. Después pone el título «Calculadora» a la ventana y la hace no redimensionable, ya que aún no es responsive.

6.7. Funcionamiento

La figura 4 muestra la calculadora al arrancar. Las figuras 5 y 6 muestran el resultado de una operación y el caso de la división entre cero.

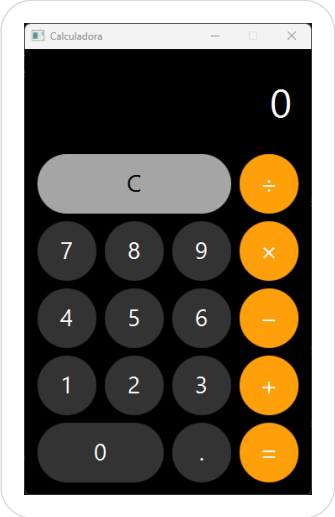


Figura 4: Implementación: calculadora al arrancar.

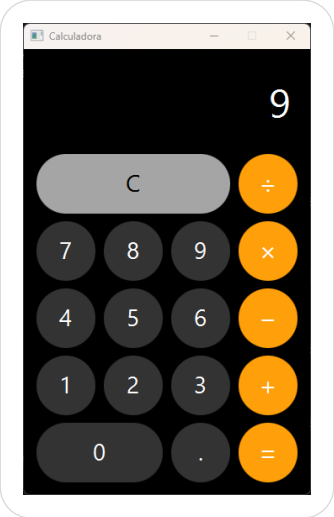


Figura 5: Implementación: resultado de una operación.

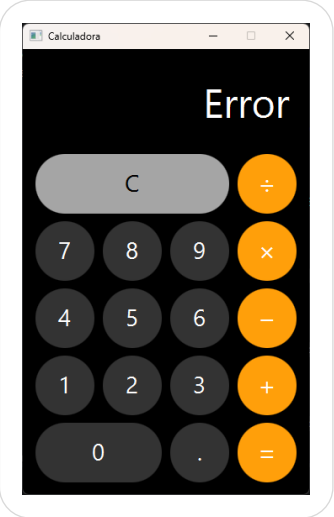


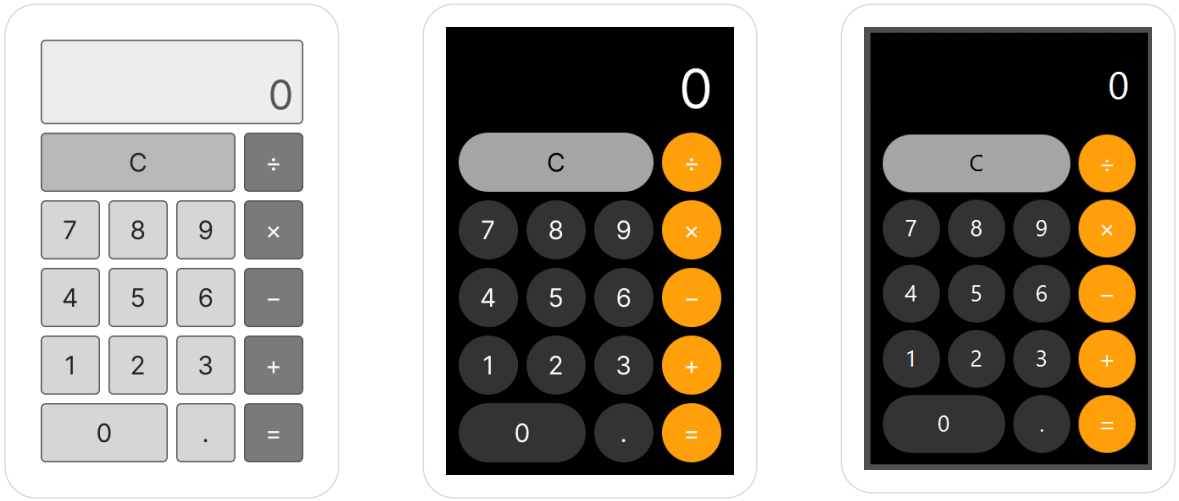
Figura 6: Implementación: división entre cero.

6.8. Limitaciones

- **Sin prioridad de operaciones.** Las operaciones encadenadas se calculan de izquierda a derecha, como en una calculadora de bolsillo. Por ejemplo, $2 + 3 \times 4$ da 20 y no 14.
- **Sin borrar un dígito ni cambiar de signo.** Solo existe `C` (o Ctrl + C), que borra todo; la tecla de retroceso no hace nada.
- **Longitud de los números.** Al escribir se admiten como mucho 10 caracteres para que el número quepa en la pantalla, pero un resultado muy grande puede no caber.
- **Redondeo.** Los resultados no enteros se muestran con un máximo de 8 decimales.
- **Distribución del teclado.** Los símbolos que se escriben con Mayúsculas (`/`, `*` y `=`) están pensados para el teclado español. En otras distribuciones se pueden usar las teclas del teclado numérico, que funcionan siempre.

7. Diferencias respecto al prototipo

La implementación es prácticamente igual al prototipo, como se ve en la figura 7. Se respeta la distribución, el orden de los números (`7 8 9` arriba), los colores y la forma de los botones. La única diferencia es la tipografía: en Figma usé otra fuente y en la implementación se usa la fuente por defecto del sistema.



Wireframe Prototipo SceneBuilder

Figura 7: Comparación entre el wireframe, el prototipo y la implementación.

8. Pruebas con JUnit

8.1. Qué se prueba

Las pruebas están en la clase `CalculadoraTest`. Hay una prueba por operación y una más para la división entre cero (cuadro 9).

Prueba	Operación	Resultado esperado
`sumar`	$5 + 3$	`8`
`restar`	$5 - 3$	`2`
`multiplicar`	5×3	`15`
`dividir`	$5 \div 2$	`2.5`
`dividirEntreCeroDaError`	$5 \div 0$	`ArithmeticException`

Cuadro 9: Pruebas de `CalculadoraTest`.

8.2. Cómo se ejecutan

Se pueden ejecutar desde el IDE o con el comando `mvn test`.

9. Documentación Javadoc

Todas las clases y métodos tienen comentarios en formato Javadoc. La documentación se genera con la herramienta Javadoc de IntelliJ IDEA (*Tools* → *Generate JavaDoc*) y se entrega junto con el código fuente.

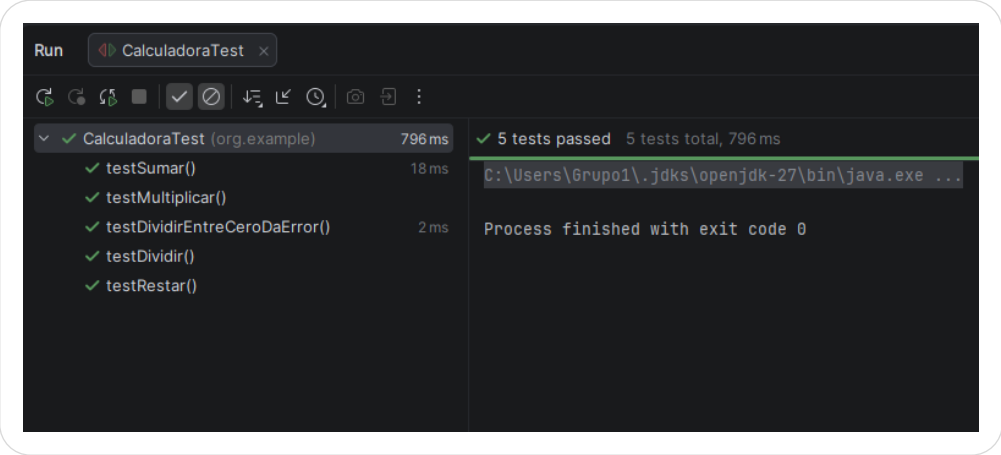


Figura 8: Resultado de las pruebas JUnit.

Para consultar la documentación, se abre el archivo `docs/index.html` generado por Javadoc con cualquier navegador web (doble clic sobre el archivo, o en IntelliJ IDEA: clic derecho → *Open In* → *Browser*).

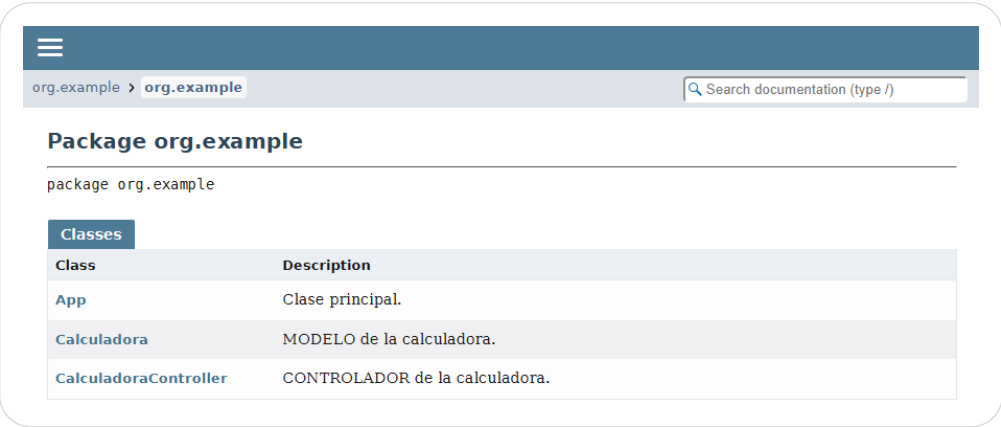


Figura 9: Documentación generada con Javadoc.

10. Código desarrollado

En esta sección se muestra el código sin los comentarios Javadoc para que sea más fácil de leer. El código entregado sí los incluye.

10.1. App.java

```
1 package org.example;
2
3 import javafx.application.Application;
4 import javafx.fxml.FXMLLoader;
5 import javafx.scene.Parent;
6 import javafx.scene.Scene;
```

```
import javafx.stage.Stage;

import java.io.IOException;

public class App extends Application {

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader loader = new FXMLLoader(App.class.getResource("calculadora.fxml"));
        Parent raiz = loader.load();
        CalculadoraController controlador = loader.getController();

        Scene escena = new Scene(raiz);

        escena.setOnKeyPressed(evento -> controlador.teclaPulsada(evento));

        stage.setTitle("Calculadora");
        stage.setResizable(false);
        stage.setScene(escena);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

10.2. Calculadora.java

```
package org.example;

public class Calculadora {

    public double sumar(double a, double b) {
        return a + b;
    }

    public double restar(double a, double b) {
        return a - b;
    }

    public double multiplicar(double a, double b) {
        return a * b;
    }

    public double dividir(double a, double b) {
        if (b == 0) {
            throw new ArithmeticException("No se puede dividir entre cero");
        }
        return a / b;
    }
}
```

10.3. CalculadoraController.java

```
package org.example;

import javafx.event.ActionEvent;
```

```java
import javafx.fxml.FXML;
import javafx.scene.Node;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyCodeCombination;
import javafx.scene.input.KeyCombination;
import javafx.scene.input.KeyEvent;

public class CalculadoraController {

    @FXML
    private Label pantalla;

    private Calculadora calculadora = new Calculadora();
    private double primerNumero = 0;
    private String operacion = "";
    private boolean numeroNuevo = true;

    private final KeyCombination combinacionBorrar =
            new KeyCodeCombination(KeyCode.C, KeyCombination.CONTROL_DOWN);

    public CalculadoraController() {
    }

    @FXML
    private void pulsarNumero(ActionEvent evento) {
        Button boton = (Button) evento.getSource();
        escribirDigito(boton.getText());
    }

    @FXML
    private void pulsarOperacion(ActionEvent evento) {
        Node boton = (Node) evento.getSource();
        elegirOperacion((String) boton.getUserData());
    }

    @FXML
    private void pulsarPunto() {
        if (numeroNuevo) {
            pantalla.setText("0.");
            numeroNuevo = false;
        } else if (!pantalla.getText().contains(".")) {
            pantalla.setText(pantalla.getText() + ".");
        }
    }

    @FXML
    private void pulsarIgual() {
        if (operacion.equals("") || pantalla.getText().equals("Error")) {
            return;
        }

        double segundoNumero = Double.parseDouble(pantalla.getText());
        double resultado = 0;

        try {
            switch (operacion) {
                case "+":
                    resultado = calculadora.sumar(primerNumero, segundoNumero);
                    break;
                case "-":
                    resultado = calculadora.restar(primerNumero, segundoNumero);
```

```
                    break;
                case "*":
                    resultado = calculadora.multiplicar(primerNumero, segundoNumero);
                    break;
                case "/":
                    resultado = calculadora.dividir(primerNumero, segundoNumero);
                    break;
            }
            pantalla.setText(formatear(resultado));
        } catch (ArithmeticException e) {
            pantalla.setText("Error");
        }

        operacion = "";
        numeroNuevo = true;
    }

    @FXML
    private void pulsarBorrar() {
        pantalla.setText("0");
        primerNumero = 0;
        operacion = "";
        numeroNuevo = true;
    }

    public void teclaPulsada(KeyEvent evento) {
        KeyCode tecla = evento.getCode();

        if (combinacionBorrar.match(evento)) {
            pulsarBorrar();
            return;
        }

        if (evento.isShiftDown()) {
            if (tecla == KeyCode.DIGIT7) {
                elegirOperacion("/");
            } else if (tecla == KeyCode.PLUS) {
                elegirOperacion("*");
            } else if (tecla == KeyCode.DIGIT0) {
                pulsarIgual();
            }
            return;
        }

        switch (tecla) {
            case DIGIT0: case NUMPAD0: escribirDigito("0"); break;
            case DIGIT1: case NUMPAD1: escribirDigito("1"); break;
            case DIGIT2: case NUMPAD2: escribirDigito("2"); break;
            case DIGIT3: case NUMPAD3: escribirDigito("3"); break;
            case DIGIT4: case NUMPAD4: escribirDigito("4"); break;
            case DIGIT5: case NUMPAD5: escribirDigito("5"); break;
            case DIGIT6: case NUMPAD6: escribirDigito("6"); break;
            case DIGIT7: case NUMPAD7: escribirDigito("7"); break;
            case DIGIT8: case NUMPAD8: escribirDigito("8"); break;
            case DIGIT9: case NUMPAD9: escribirDigito("9"); break;

            case ADD: case PLUS:       elegirOperacion("+"); break;
            case SUBTRACT: case MINUS: elegirOperacion("-"); break;
            case MULTIPLY:             elegirOperacion("*"); break;
            case DIVIDE:               elegirOperacion("/"); break;

            case DECIMAL: case PERIOD: case COMMA: pulsarPunto(); break;

```

```
            case ENTER: case EQUALS:   pulsarIgual(); break;

            default: break;
        }
    }

    private void escribirDigito(String digito) {
        if (numeroNuevo || pantalla.getText().equals("0")) {
            pantalla.setText(digito);
            numeroNuevo = false;
        } else if (pantalla.getText().length() < 10) {
            pantalla.setText(pantalla.getText() + digito);
        }
    }

    private void elegirOperacion(String simbolo) {
        if (pantalla.getText().equals("Error")) {
            return;
        }

        if (!operacion.equals("") && !numeroNuevo) {
            pulsarIgual();
            if (pantalla.getText().equals("Error")) {
                return;
            }
        }

        operacion = simbolo;
        primerNumero = Double.parseDouble(pantalla.getText());
        numeroNuevo = true;
    }

    private String formatear(double resultado) {
        if (resultado == (long) resultado) {
            return String.valueOf((long) resultado);
        }

        resultado = Math.round(resultado * 100000000) / 100000000.0;
        return String.valueOf(resultado);
    }
}
```

10.4. calculadora.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.ColumnConstraints?>
<?import javafx.scene.layout.GridPane?>
<?import javafx.scene.layout.RowConstraints?>
<?import javafx.scene.layout.VBox?>

<VBox prefHeight="530.0" prefWidth="340.0" spacing="10.0" style="-fx-background-color:
  #000000;" xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
  fx:controller="org.example.CalculadoraController">
    <padding>
        <Insets bottom="15.0" left="15.0" right="15.0" top="15.0" />
    </padding>
```

```
15      <children>
16          <Label fx:id="pantalla" alignment="CENTER_RIGHT" maxWidth="Infinity" prefHeight="
    100.0" style="-fx-text-fill: #FFFFFF; -fx-font-size: 48px; -fx-padding: 0 8 0 0;" text="0
    " />
17
18          <GridPane hgap="10.0" vgap="10.0">
19              <columnConstraints>
20                  <ColumnConstraints prefWidth="70.0" />
21                  <ColumnConstraints prefWidth="70.0" />
22                  <ColumnConstraints prefWidth="70.0" />
23                  <ColumnConstraints prefWidth="70.0" />
24              </columnConstraints>
25              <rowConstraints>
26                  <RowConstraints prefHeight="70.0" />
27                  <RowConstraints prefHeight="70.0" />
28                  <RowConstraints prefHeight="70.0" />
29                  <RowConstraints prefHeight="70.0" />
30                  <RowConstraints prefHeight="70.0" />
31              </rowConstraints>
32              <children>
33                  <Button text="C" prefWidth="230" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarBorrar" style="-fx-background-color: #A5A5A5; -
    fx-text-fill: #000000; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="0" GridPane.rowIndex="0" GridPane.columnSpan="3" />
34                  <Button text="÷" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarOperacion" style="-fx-background-color: #FF9F0A
    ; -fx-text-fill: #FFFFFF; -fx-font-size: 32px; -fx-background-radius: 35;" userData="/"
    GridPane.columnIndex="3" GridPane.rowIndex="0" />
35                  <Button text="7" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="0" GridPane.rowIndex="1" />
36                  <Button text="8" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="1" GridPane.rowIndex="1" />
37                  <Button text="9" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="2" GridPane.rowIndex="1" />
38                  <Button text="×" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarOperacion" style="-fx-background-color: #FF9F0A
    ; -fx-text-fill: #FFFFFF; -fx-font-size: 32px; -fx-background-radius: 35;" userData="*"
    GridPane.columnIndex="3" GridPane.rowIndex="1" />
39                  <Button text="4" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="0" GridPane.rowIndex="2" />
40                  <Button text="5" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="1" GridPane.rowIndex="2" />
41                  <Button text="6" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="2" GridPane.rowIndex="2" />
42                  <Button text="−" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarOperacion" style="-fx-background-color: #FF9F0A
    ; -fx-text-fill: #FFFFFF; -fx-font-size: 32px; -fx-background-radius: 35;" userData="-"
    GridPane.columnIndex="3" GridPane.rowIndex="2" />
43                  <Button text="1" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
```

```
    columnIndex="0" GridPane.rowIndex="3" />
44                <Button text="2" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="1" GridPane.rowIndex="3" />
45                <Button text="3" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="2" GridPane.rowIndex="3" />
46                <Button text="+" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarOperacion" style="-fx-background-color: #FF9F0A
    ; -fx-text-fill: #FFFFFF; -fx-font-size: 32px; -fx-background-radius: 35;" userData="+"
    GridPane.columnIndex="3" GridPane.rowIndex="3" />
47                <Button text="0" prefWidth="150" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarNumero" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="0" GridPane.rowIndex="4" GridPane.columnSpan="2" />
48                <Button text="." prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarPunto" style="-fx-background-color: #333333; -
    fx-text-fill: #FFFFFF; -fx-font-size: 28px; -fx-background-radius: 35;" GridPane.
    columnIndex="2" GridPane.rowIndex="4" />
49                <Button text="=" prefWidth="70" prefHeight="70" mnemonicParsing="false"
    focusTraversable="false" onAction="#pulsarIgual" style="-fx-background-color: #FF9F0A; -
    fx-text-fill: #FFFFFF; -fx-font-size: 32px; -fx-background-radius: 35;" GridPane.
    columnIndex="3" GridPane.rowIndex="4" />
50            </children>
51        </GridPane>
52    </children>
53 </VBox>
```

10.5. CalculadoraTest.java

```
1  package org.example;
2
3  import static org.junit.jupiter.api.Assertions.assertEquals;
4  import static org.junit.jupiter.api.Assertions.assertThrows;
5
6  import org.junit.jupiter.api.Test;
7
8  class CalculadoraTest {
9
10     private Calculadora calculadora = new Calculadora();
11
12     @Test
13     void testSumar() {
14         assertEquals(8, calculadora.sumar(5, 3));
15     }
16
17     @Test
18     void testRestar() {
19         assertEquals(2, calculadora.restar(5, 3));
20     }
21
22     @Test
23     void testMultiplicar() {
24         assertEquals(15, calculadora.multiplicar(5, 3));
25     }
26
27     @Test
28     void testDividir() {
```

```
29          assertEquals(2.5, calculadora.dividir(5, 2));
30      }
31
32      @Test
33      void testDividirEntreCeroDaError() {
34          assertThrows(ArithmeticException.class, () -> calculadora.dividir(5, 0));
35      }
36  }
```